BRAC University – CSE 221: Algorithms

Semester Final Exam – Fall 2022 | Complete Solutions (Set A & Set B)

Note: The exam has two sets (A and B). Where they differ, both versions are solved. Set A and Set B share the same question structure but with different edge weights / frequencies / intervals.

Question 1(a) – Shortest Path from Simon to Every Player [6 marks]

The problem asks for the minimum-cost path from Simon (source) to every other player. Since all edge weights are positive, **Dijkstra's Algorithm** is the optimal choice.

■ *Set A – Edge List*

From	To	Cost
Simon	Rodri	2
Simon	Cesar	4
Rodri	Cesar	1
Rodri	Pedri	7
Cesar	Gavi	3
Gavi	Pedri	2
Pedri	Morata	1
Gavi	Morata	5

Step-by-step Dijkstra Execution (Set A)

Initialise: dist[Simon]=0, all others=∞. Unvisited = {Simon, Rodri, Cesar, Pedri, Gavi, Morata}

Step	Visited Node	Dist Updates	dist[] after step
1	Simon (dist=0)	Rodri: 0+2=2 Cesar: 0+4=4	Simon=0, Rodri=2, Cesar=4, Pedri=∞, Gavi=∞, Morata=∞
2	Rodri (dist=2, min unvisited)	Cesar: min(4, 2+1)=3 Pedri: min(∞, 2+7)=9	Simon=0, Rodri=2, Cesar=3, Pedri=9, Gavi=∞, Morata=∞
3	Cesar (dist=3, min unvisited)	Gavi: min(∞, 3+3)=6	Simon=0, Rodri=2, Cesar=3, Pedri=9, Gavi=6, Morata=∞
4	Gavi (dist=6, min unvisited)	Pedri: min(9, 6+2)=8 Morata: min(∞, 6+5)=11	Simon=0, Rodri=2, Cesar=3, Pedri=8, Gavi=6, Morata=11
5	Pedri (dist=8, min unvisited)	Morata: min(11, 8+1)=9	Simon=0, Rodri=2, Cesar=3, Pedri=8, Gavi=6, Morata=9
6	Morata (dist=9, min unvisited)	No outgoing edges	Simon=0, Rodri=2, Cesar=3, Pedri=8, Gavi=6, Morata=9

Final Shortest Distances (Set A):

Player	Shortest Distance from Simon	Path
Simon	0	Simon
Rodri	2	Simon → Rodri
Cesar	3	Simon → Rodri → Cesar
Gavi	6	Simon → Rodri → Cesar → Gavi
Pedri	8	Simon → Rodri → Cesar → Gavi → Pedri

Morata	9	Simon → Rodri → Cesar → Gavi → Pedri → Morata

■ *Set B – Edge List*

From	To	Cost
Simon	Cesar	2
Simon	Rodri	4
Cesar	Rodri	1
Rodri	Pedri	3
Cesar	Gavi	7
Pedri	Gavi	2
Pedri	Morata	5
Gavi	Morata	1

Step-by-step Dijkstra Execution (Set B)

Initialise: dist[Simon]=0, all others=∞. Unvisited = {Simon, Rodri, Cesar, Pedri, Gavi, Morata}

Step	Visited Node	Dist Updates	dist[] after step
1	Simon (dist=0)	Cesar: 0+2=2 Rodri: 0+4=4	Simon=0, Cesar=2, Rodri=4, Pedri=∞, Gavi=∞, Morata=∞
2	Cesar (dist=2, min unvisited)	Rodri: min(4, 2+1)=3 Gavi: min(∞, 2+7)=9	Simon=0, Cesar=2, Rodri=3, Pedri=∞, Gavi=9, Morata=∞
3	Rodri (dist=3, min unvisited)	Pedri: min(∞, 3+3)=6	Simon=0, Cesar=2, Rodri=3, Pedri=6, Gavi=9, Morata=∞
4	Pedri (dist=6, min unvisited)	Gavi: min(9, 6+2)=8 Morata: min(∞, 6+5)=11	Simon=0, Cesar=2, Rodri=3, Pedri=6, Gavi=8, Morata=11
5	Gavi (dist=8, min unvisited)	Morata: min(11, 8+1)=9	Simon=0, Cesar=2, Rodri=3, Pedri=6, Gavi=8, Morata=9
6	Morata (dist=9, min unvisited)	No outgoing edges	Simon=0, Cesar=2, Rodri=3, Pedri=6, Gavi=8, Morata=9

Final Shortest Distances (Set B):

Player	Shortest Distance from Simon	Path
Simon	0	Simon
Cesar	2	Simon → Cesar
Rodri	3	Simon → Cesar → Rodri
Pedri	6	Simon → Cesar → Rodri → Pedri
Gavi	8	Simon → Cesar → Rodri → Pedri → Gavi
Morata	9	Simon → Cesar → Rodri → Pedri → Gavi → Morata

Question 1(b) – Largest Group (Strongly Connected Components) [4 marks]

We need the largest group where every player can pass to every other — this is a **Strongly Connected Component (SCC)**. We use **Kosaraju's Algorithm**: (1) Run DFS on original graph and record finish times, (2) Transpose the graph, (3) Run DFS on transposed graph in decreasing finish-time order. Each DFS tree in step 3 is one SCC.

■ *Set A – Directed Edges*

Edges (Set A)
Simon→Rodri, Simon→Cesar, Rodri→Busquets, Rodri→Pedri, Cesar→Rodri, Busquets→Gavi, Busquets→Pedri, Gavi→Cesar, Gavi→Pedri, Gavi→Fati, Fati→Morata, Pedri→Fati, Morata→Pedri

Step 1 – DFS on original graph (Set A), record finish order:

Start DFS from Simon (arbitrary). Track discovery/finish times:
DFS path: Simon→Rodri→Busquets→Gavi→Cesar (back to Rodri via Cesar→Rodri, finish Cesar), Gavi→Pedri→Fati→Morata→Pedri (already visited), finish Morata, finish Fati, finish Pedri, finish Gavi, finish Busquets, finish Rodri, finish Simon.

Finish order (last finished first for step 3): Cesar, Morata, Fati, Pedri, Gavi, Busquets, Rodri, Simon

Step 2 – Transposed Graph (Set A) — reverse all edges:

```
Rodri→Simon, Cesar→Simon, Busquets→Rodri, Pedri→Rodri, Rodri→Cesar,
Gavi→Busquets, Pedri→Busquets, Cesar→Gavi, Pedri→Gavi, Fati→Gavi, Morata→Fati,
Fati→Pedri, Pedri→Morata
```

Step 3 – DFS on transposed graph in reverse finish order:

DFS Start	Nodes Reachable	SCC
Simon	Simon only (Rodri→Simon leads back but Simon already done)	{Simon}
Rodri	Rodri→Cesar→Gavi→Busquets→Rodri (cycle!)	{Rodri, Cesar, Gavi, Busquets}
Pedri	Pedri→Morata→Fati→Pedri (cycle!)	{Pedri, Morata, Fati}

■ **Largest SCC (Set A) = {Rodri, Cesar, Gavi, Busquets}** — size 4.
Verification: Rodri→Busquets→Gavi→Cesar→Rodri forms a cycle, so all 4 can pass among themselves.

■ *Set B – Directed Edges*

Edges (Set B)
Simon→Cesar, Simon→Rodri, Cesar→Gavi, Gavi→Busquets, Gavi→Pedri, Gavi→Fati, Busquets→Pedri, Busquets→Rodri, Pedri→Morata, Fati→Pedri, Morata→Fati, Rodri→Pedri, Rodri→Cesar

Step 1 – DFS on original graph (Set B), finish order:

DFS from Simon: Simon→Cesar→Gavi→Busquets→Pedri→Morata→Fati→Pedri(visited)→finish Fati→finish Morata→finish Pedri→Busquets→Rodri→Cesar(visited)→Pedri(visited)→finish Rodri→finish Busquets→Gavi→Pedri(visited)→Fati(visited)→finish Gavi→finish Cesar→Simon→Rodri(visited)→finish Simon

Finish order (last to finish = processed first in step 3): Fati, Morata, Pedri, Rodri, Busquets, Gavi,

Cesar, Simon

Step 2 – Transposed Graph (Set B):

```
Cesar→Simon, Rodri→Simon, Gavi→Cesar, Busquets→Gavi, Pedri→Gavi, Fati→Gavi,
Pedri→Busquets, Rodri→Busquets, Morata→Pedri, Pedri→Fati, Fati→Morata,
Pedri→Rodri, Cesar→Rodri
```

Step 3 – DFS on transposed graph in reverse finish order:

DFS Start	Nodes Reachable in Transposed	SCC
Simon	Simon only	{Simon}
Cesar	Cesar→Simon(done)→Rodri→Simon(done)→Busquets→Gavi→Cesar(done) — cycle!	{Cesar, Rodri, Busquets, Gavi}
Pedri	Pedri→Fati→Morata→Pedri — cycle!	{Pedri, Fati, Morata}

■ **Largest SCC (Set B) = {Cesar, Rodri, Busquets, Gavi}** — size 4.
Verification: Cesar→Gavi→Busquets→Rodri→Cesar forms a cycle, so all 4 can pass among themselves.

Question 2(a) – Huffman Coding [4 + 1 marks]

■ *Set A: Symbols A=40, B=10, C=20, D=15, +=15*

(i) Building the Huffman Tree

Step	Action / Min-heap state	Result
Initial	Priority queue (min-heap): B=10, D=15, +=15, C=20, A=40	—
Step 1	Extract two minimums: B(10) and D(15) Merge → internal node N1=25 Push N1 back	N1(25): left=B(10), right=D(15)
Step 2	Heap: +(15), C(20), N1(25), A(40) Extract +(15) and C(20) Merge → N2=35	N2(35): left=+(15), right=C(20)
Step 3	Heap: N1(25), A(40), N2(35) Extract N1(25) and N2(35) Merge → N3=60	N3(60): left=N1(25), right=N2(35)
Step 4	Heap: A(40), N3(60) Extract A(40) and N3(60) Merge → ROOT=100	ROOT(100): left=A(40), right=N3(60)

Huffman Tree Structure (Set A):

Tree Structure (Set A)

```
ROOT(100)
■■■ 0: A (40)          → Code: 0
■■■ 1: N3(60)
    ■■■ 0: N1(25)
    ■   ■■■ 0: B(10)   → Code: 100
    ■   ■■■ 1: D(15)   → Code: 101
    ■■■ 1: N2(35)
        ■■■ 0: +(15)   → Code: 110
        ■■■ 1: C(20)   → Code: 111
```

Symbol	Frequency	Huffman Code	Code Length	Contribution (freq × len)
A	40	0	1	40
B	10	100	3	30
C	20	111	3	60
D	15	101	3	45
+	15	110	3	45
Total	**100**	**—**	**—**	**220 bits**

(ii) Decode: 100010111001010 (Set A)

Using codes: A=0, B=100, C=111, D=101, +=110

Bits consumed	Match	Decoded symbol	Remaining bits
1	0	A	00010111001010
1	0	A	0010111001010
1	0	A	010111001010

1	0	A	10111001010
3	101	D	11001010
3	110	+	01010
1	0	A	1010
3	101	D	0
1	0	A	—

■ Decoded message (Set A): A A A A D + A D A

■ *Set B: Symbols A=35, B=15, C=25, D=10, #=20*

(i) Building the Huffman Tree

Step	Action / Min-heap state	Result
Initial	Priority queue: D=10, B=15, #=20, C=25, A=35	—
Step 1	Extract D(10) and B(15) Merge → N1=25	N1(25): left=D(10), right=B(15)
Step 2	Heap: #(20), N1(25), C(25), A(35) Extract #(20) and N1(25) Merge → N2=45	N2(45): left=#(20), right=N1(25)
Step 3	Heap: C(25), A(35), N2(45) Extract C(25) and A(35) Merge → N3=60	N3(60): left=C(25), right=A(35)
Step 4	Heap: N2(45), N3(60) Extract N2(45) and N3(60) Merge → ROOT=105	ROOT(105): left=N2(45), right=N3(60)

Huffman Tree Structure (Set B):

Tree Structure (Set B)

```
ROOT(105)
■■■ 0: N2(45)
■   ■■■ 0: #(20)          → Code: 00
■   ■■■ 1: N1(25)
■       ■■■ 0: D(10)      → Code: 010
■       ■■■ 1: B(15)      → Code: 011
■■■ 1: N3(60)
    ■■■ 0: C(25)           → Code: 10
    ■■■ 1: A(35)           → Code: 11
```

Symbol	Frequency	Huffman Code	Code Length	Contribution
A	35	11	2	70
B	15	011	3	45
C	25	10	2	50
D	10	010	3	30
#	20	00	2	40
Total	**105**	**—**	**—**	**235 bits**

(ii) Decode: 100010111001010 (Set B)

Using codes: A=11, B=011, C=10, D=010, #=00

Bits consumed	Bits read	Decoded symbol	Remaining bits
2	10	C	0010111001010
2	00	#	10111001010
2	10	C	111001010
2	11	A	1001010
2	10	C	01010
3	010	D	10
2	10	C	—

■ **Decoded message (Set B): C # C A C D C**

Question 2(b) – Train Scheduling (Interval Scheduling) [3 + 2 marks]

Part (i): Maximum trains using ONE platform — Greedy algorithm: sort by **earliest deadline (departure time)**, then greedily pick trains that don't overlap. **Part (ii): Minimum platforms needed** — equivalent to the maximum number of trains that overlap at any single point in time.

■ *Set A: {[8,13), [6,9), [11,14), [2,7), [1,7), [12,20), [7,13), [13,20)}*

Part (i) – Sort by departure, greedily pick:

Train	Arrival	Departure	Selected?	Reason
E: [1,7)	1	7	✓ YES	First train picked
D: [2,7)	2	7	✗ NO	Overlaps with E (departs 7, arrives 2 < 7)
B: [6,9)	6	9	✗ NO	Overlaps with E (departs 7, arrives 6 < 7)
A: [8,13)	8	13	✓ YES	Starts at 8 ≥ 7 (end of E), no conflict
G: [7,13)	7	13	✗ NO	Starts at 7 = 7, but A already chosen with end 13
C: [11,14)	11	14	✗ NO	Overlaps with A (11 < 13)
F: [12,20)	12	20	✗ NO	Overlaps with A (12 < 13)
H: [13,20)	13	20	✓ YES	Starts at 13 ≥ 13 (end of A), no conflict

■ **Maximum trains on 1 platform (Set A) = 3**: [1,7), [8,13), [13,20)

Part (ii) – Minimum platforms needed (Set A):

Create arrival and departure events, sort them, sweep through counting overlaps:
Events in order: Arr1, Arr2, Arr6(2), Dep4(7), Dep5(7), Dep2(9)... count simultaneous trains peak.

```
Arrivals: 1,2,6,7,8,11,12,13 | Departures: 7,7,9,13,13,14,20,20
Sweep: +1(arr1)=1, +1(arr2)=2, +1(arr6)=3, -1(dep7)=2, -1(dep7)=1, +1(arr7)=2,
+1(arr8)=3, -1(dep9)=2, +1(arr11)=3, +1(arr12)=4 ← PEAK, -1(dep13)=3, +1(arr13)=4 ←
PEAK, -1(dep13)=3, -1(dep14)=2, -1(dep20)=1, -1(dep20)=0
```

■ **Minimum platforms needed (Set A) = 4**

■ *Set B: {[8,12), [6,9), [11,14), [2,7), [1,7), [12,20), [7,12), [13,19)}*

Part (i) – Sort by departure, greedily pick:

Train	Arrival	Departure	Selected?	Reason
E: [1,7)	1	7	✓ YES	First train picked
D: [2,7)	2	7	✗ NO	Overlaps with E
B: [6,9)	6	9	✗ NO	Overlaps with E
A: [8,12)	8	12	✓ YES	Starts at 8 ≥ 7, no conflict
G: [7,12)	7	12	✗ NO	Starts at 7=7 but A already chosen
F: [12,20)	12	20	✓ YES	Starts at 12 ≥ 12 (end of A), no conflict
C: [11,14)	11	14	✗ NO	Overlaps with A and F
H: [13,19)	13	19	✗ NO	Overlaps with F

■ **Maximum trains on 1 platform (Set B) = 3**: [1,7), [8,12), [12,20)

Part (ii) – Minimum platforms needed (Set B):

```
Arrivals: 1,2,6,7,8,11,12,13 | Departures: 7,7,9,12,12,14,19,20
Sweep: +1=1(1), +1=2(2), +1=3(6), -1=2(7), -1=1(7), +1=2(7), +1=3(8), -1=2(9),
+1=3(11), +1=4(12) ← PEAK, -1=3(12), -1=2(12), +1=3(13), -1=2(14), -1=1(19),
-1=0(20)
```

■ Minimum platforms needed (Set B) = 4

Question 3 – Coin Change using Dynamic Programming [6 + 2 + 2 marks] (Set A)

Problem Setup:

Plushie price = 700 BDT. VAT = 6% → Total = 700 × 1.06 = **742 BDT**.
Customer paid 750 BDT → Change = 750 – 742 = **8 BDT**.
Available coins: 1 taka, 2 taka, 5 taka. Minimize number of coins to make 8 BDT.

(i) DP Table Construction

Let dp[i] = minimum coins needed to make amount i. dp[0]=0, dp[i]=∞ initially.
Recurrence: dp[i] = min(dp[i], dp[i – coin] + 1) for each coin in {1, 2, 5}

Amount i →	0	1	2	3	4	5	6	7	8
dp[i] (min coins)	0	1	1	2	2	1	2	2	**3**

■ **dp[8] = 2** — The cashier should return **2 coins** for 8 BDT change.

(ii) Traceback – Which coins?

Start at dp[8]=2. Try subtracting each coin to find which was used:

Current Amount	Try coin	Check dp[amount-coin]	Decision
8	5	dp[3] = 3 → dp[8-5]+1 = 4 ≠ 2	Not via 5
8	2	dp[6] = 3 → dp[8-2]+1 = 4 ≠ 2	Not via 2
8	1	dp[7] = 1 → dp[8-1]+1 = 2 = dp[8] ✓	Used coin 1? Check further...
			Actually: dp[8]=2, try 5: dp[3]=3 no; try 2: dp[6]=3 no
7	5	dp[2]=1 → dp[7-5]+1=2 = dp[7] ✓	✓ Use coin 5 taka
2	2	dp[0]=0 → dp[2-2]+1=1 = dp[2] ✓	✓ Use coin 2 taka

■ **Coins returned: one 5-taka coin + one 2-taka coin = 7? Wait — let us verify directly.**
Optimal for 8 BDT = 5 + 2 + 1 = 3 coins, OR = 5 + 1 + 1 + 1 = 4? Let us recheck: dp[8]: coin5 → dp[3]+1; dp[3]: coin2→dp[1]+1=2, coin1→dp[2]+1=2 → dp[3]=2; so dp[8] via coin5 = 3.
coin2→dp[6]+1; dp[6]: coin5→dp[1]+1=2, coin2→dp[4]+1; dp[4]=2(2+2); so dp[6]=3; dp[8] via coin2=4.
coin1→dp[7]+1; dp[7]: coin5→dp[2]+1=2, so dp[7]=2; dp[8] via coin1=3.
So dp[8]=3. Coins: 5+2+1 = 3 coins.

Corrected DP Table:

Amount i →	0	1	2	3	4	5	6	7	8
dp[i] (min coins)	0	1	1	2	2	1	2	2	**3**

Correct dp values: dp[0]=0, dp[1]=1, dp[2]=1, dp[3]=2, dp[4]=2, dp[5]=1, dp[6]=2, dp[7]=2, dp[8]=3
■ **dp[8] = 3 coins. Coins returned: 5 taka + 2 taka + 1 taka = 8 BDT**

(iii) Time Complexity of Brute Force (No Memoization):

In the brute-force recursive approach, at each step we try every coin denomination. For a target amount n with k coin types, the recurrence is:

T(n) = k × T(n – 1) in the worst case (when smallest coin = 1)

This gives **T(n) = O(k^n)**. For our problem: k=3, n=8, so O(3^8) = O(6561).

More precisely: The recursion tree has branching factor k=3 (three coin choices) and depth n=8 (we subtract at least 1 each call). Without memoization, the same subproblems are recomputed exponentially many times. Hence the brute force time complexity is **$O(k^n)$** = **Exponential**.

Question 3 – 0/1 Knapsack (Denver's DP) + Greedy (Nairobi) [8 + 2 marks] (Set B)

Problem Setup:

Item	Jewelry	Sculpture	Painting	Book	Mummy
	5	9	5	4	6
Weight (kg)	3	5	4	1	12
	1.67	1.80	1.25	4.00	0.50

Capacity W = 8 kg. Items: Jewelry(J)=3kg/$5, Sculpture(Sc)=5kg/$9, Painting(Pa)=4kg/$5, Book(Bo)=1kg/$4, Mummy(Mu)=12kg/$6

(i) Denver's DP Approach – 0/1 Knapsack Table

dp[i][w] = maximum profit using first i items with capacity w.
Items ordered: 1=Jewelry(3,5), 2=Sculpture(5,9), 3=Painting(4,5), 4=Book(1,4), 5=Mummy(12,6)

Item \ Cap	0	1	2	3	4	5	6	7	8
	0	0	0	0	0	0	0	0	0
1:Jewelry	0	0	0	5	5	5	5	5	5
	0	0	0	5	5	9	9	9	14
3:Paint	0	0	0	5	5	9	9	10	14
	0	4	4	5	9	9	13	13	14
5:Mummy	0	4	4	5	9	9	13	13	**14**

■ **Maximum profit (Denver) = dp[5][8] = 14 dollars**

Traceback — Which items did Denver pick?

Step	i (item)	w (remaining cap)	dp[i][w] vs dp[i-1][w]	Action
1	5: Mummy	8	dp[5][8]=14 = dp[4][8]=14	✗ SKIP
2	4: Book	8	dp[4][8]=14 = dp[3][8]=14	✗ SKIP
3	3: Painting	8	dp[3][8]=14 = dp[2][8]=14	✗ SKIP
4	2: Sculpture	8	dp[2][8]=14 ≠ dp[1][8]=5	✓ TAKE Sculpture (w-=5)
5	1: Jewelry	3	dp[1][3]=5 ≠ dp[0][3]=0	✓ TAKE Jewelry (w-=3)

■ **Denver picks: Jewelry, Sculpture | Total profit = $14 | Total weight = 8 kg**

(ii) Nairobi's Greedy Approach – Is it valid?

Nairobi sorts by **value-to-weight ratio** (greedy) and picks items:
Ratios: Book=4.00, Sculpture=1.80, Jewelry=1.67, Painting=1.25, Mummy=0.50

Order	Item	Profit	Weight	V/W Ratio	Remaining Cap	Pick?
1st	Book	$4	1 kg	4.00	8→7	✓ YES
2nd	Sculpture	$9	5 kg	1.80	7→2	✓ YES
3rd	Jewelry	$5	3 kg	1.67	2 (need 3)	✗ NO (overweight)
4th	Painting	$5	4 kg	1.25	2 (need 4)	✗ NO (overweight)

5th	Mummy	$6	12 kg	0.50	2 (need 12)	✗ NO (overweight)

Nairobi's greedy result: Book + Sculpture = $4 + $9 = **$13**

Denver's optimal (DP): Jewelry, Sculpture = **$14**

■ **Nairobi's belief is INVALID.**
The greedy approach yields $13, while the optimal DP gives $14. The greedy approach fails for 0/1 Knapsack because items cannot be split — taking a high-ratio item (Sculpture at 5 kg) may block combinations of smaller items that collectively yield higher profit. Greedy only guarantees optimality for the **Fractional Knapsack** problem, not the 0/1 variant.

Answer Summary

Q	Part	Set A Answer	Set B Answer	Marks
1a	Dijkstra shortest paths	Simon=0, Rodri=2, Cesar=3, Gavi=6, Pedri=8, Morata=9	Simon=0, Cesar=2, Rodri=3, Pedri=6, Gavi=8, Morata=9	6
1b	Largest SCC (Kosaraju's)	{Rodri, Cesar, Gavi, Busquets} size = 4	{Cesar, Rodri, Busquets, Gavi} size = 4	4
2a(i)	Huffman codes	A=0, B=100, C=111, D=101, +=110	A=11, B=011, C=10, D=010, #=00	4
2a(ii)	Decode 100010111001010	A A A A D + A D A	C # C A C D C	1
2b(i)	Max trains (1 platform)	3 trains: [1,7), [8,13), [13,20)	3 trains: [1,7), [8,12), [12,20)	3
2b(ii)	Min platforms needed	4 platforms	4 platforms	2
3(i)	DP answer	Change=8 BDT → 3 coins (5+2+1)	Denver max profit=$14 Items: Jewelry, Sculpture	6/8
3(ii)	Traceback	Coins: 5-taka, 2-taka, 1-taka	DP table traceback shown	2
3(iii)	Complexity / Nairobi	Brute force = O(k^n) exponential	Nairobi WRONG: greedy gives $13 < DP's $14	2

Prepared as a complete solution guide for CSE 221 Final Exam Fall 2022 (Set A & Set B). All steps shown as required by the examiner.